CHAPTER 09 DATA ENGINEERING / SNOWPARK PYTHON

Data Engineering with Snowpark Python

Write transformations in Python without losing the data platform

A correct transformation written in the wrong place can still be a bad design. The Finland Energy Analytics service receives a wind correction. The incoming event has a trusted business key and revision, but a junior engineer copies the entire bronze table to a laptop, filters the data in pandas, and uploads the result again. The calculation is right for six records. At national scale, the same pattern transfers sensitive rows across a trust boundary, creates an unbounded memory dependency, and makes retries harder to audit. There is a better question than "Could Python express this calculation?" It is "Where should the calculation execute, and what proof of correctness survives deployment?"

Chapters 4–8 established data modelling, batch loading, JSON processing, delivery semantics and incremental state. Snowpark adds a *Python interface to relational work* while preserving Snowflake's ability to execute supported operations near its data. It does not repeal the rules of secure access, deterministic business keys, idempotent updates or measured cost.

Learning outcomes

After this chapter, you can (1) explain the difference between Python client orchestration and server-side execution; (2) manage least-privilege sessions; (3) build and inspect lazy DataFrame transformations; (4) evaluate joins, casts and aggregates; (5) decide between SQL, UDFs and stored procedures; (6) test without incurring cloud charges; and (7) design a maintainable, observable Snowpark pipeline.

Prerequisites. Chapters 3–6 and competent basic Python. SQL examples and production integration require access to a permitted Snowflake account. The bundled *pure-Python offline fixture* runs on Python 3.10+ without network access; a Snowpark integration run is not claimed.

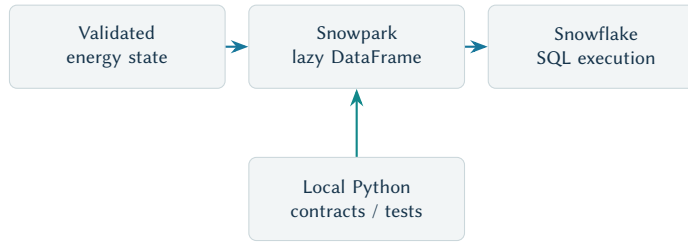


Figure 9.1. Local control code defines a plan; supported relational work runs on Snowflake compute.

9.1 Where Snowpark fits in the platform

9.1.1 Snowpark's role in data engineering

Python has two distinct roles in an analytical platform: it can *orchestrate* work and it can *express* transformations. Snowpark provides a DataFrame API that lets Python code describe projections, filters, joins, windows and aggregations over Snowflake tables. For operations supported by the DataFrame API, the engine generates SQL and executes the relational plan in Snowflake. The Python process is not ordinarily reading every row to evaluate a `filter` expression itself. This division is both the central power of Snowpark and the source of many misconceptions.

Imagine an energy table with one approved row per provider, technology and UTC interval. To compute an hourly total, SQL is an excellent fit. A Snowpark DataFrame can express the same logic while sharing names, parameters and reusable business helpers with a broader Python codebase. The platform still performs the scan and aggregation. Snowpark is not a requirement to do analytics correctly; it is a reasonable interface when programmatic composition is valuable.

Listing 1: Declarative query construction; cloud execution is not performed here.

```
from snowflake.snowpark.functions import col, sum as sf_sum
energy = session.table('ENERGY_SILVER')
plan = (energy.filter(col('ENERGY_MWH') >= 0)
        .group_by('INTERVAL_UTC')
        .agg(sf_sum('ENERGY_MWH').alias('TOTAL_MWH')))
# plan is a DataFrame, not a list of downloaded rows.
```

Why it matters. A transformation function can receive a session and return a DataFrame without writing a table, triggering data transfer, or choosing the consumer's schema. That separation makes a pipeline testable and lets an orchestrator own when an action should happen.

Architectural decision
Use Snowpark when Python organisation, complex reusable expression-building, or Python runtime integration provides concrete value. If a small, stable SQL model already expresses the business rule transparently, introducing a Python application and dependencies may increase maintenance without improving correctness. Compare models against the same acceptance tests before introducing an extra runtime.

Project reference: P094: Getting started with Snowpark Python. *Not independently executed; rights review pending.*

Check. Is a DataFrame returned by `session.table()` an in-memory pandas DataFrame? Why does the distinction matter when a source table grows one hundredfold?

9.1.2 Python execution boundaries

A Snowpark program crosses more than one boundary. The *client* authenticates, creates a Session and constructs a logical DataFrame. *Virtual warehouse compute* ordinarily executes supported query plans generated from that DataFrame. Python code inside a registered *user-defined function* (UDF) or *stored procedure* runs in a Snowflake-managed Python environment, subject to its applicable runtime and sandbox restrictions. These environments have different deployment, permissions and package availability; describing all three simply as "Python runs in Snowflake" obscures important operational detail.

Boundary	Typical action	Risk to control
Client process	Construct DataFrame, call actions	Credential handling, accidental data collection
Snowflake SQL engine	Filter, join, aggregate	Compute consumption, scan volume, permissions
Python UDF/runtime	Execute Python handler	Package, sandbox, row processing overhead
External compute	Process exported data	Network exposure, duplication, transfer cost

Table 9.1. Execution location determines data, security and cost obligations.

The code below *looks* local, but the expensive relational operation belongs to a virtual warehouse when an action is called. Conversely, `collect()` returns a *local list* and may materialise many rows in client memory. `to_pandas()` similarly transfers a result to the Python process. Protect these actions with a narrow selection, a row limit for inspection, and explicit assertions about expected cardinality.

Listing 2: Safe inspection versus unbounded client materialisation.

```
preview = plan.sort('INTERVAL_UTC').limit(5).collect()
# CAUTION: do not call plan.collect() on an unbounded dataset.
```

A Python worksheet in Snowsight, a notebook, a command-line application and a stored procedure may all use Snowpark, but their session ownership and deployment semantics differ. In a Python worksheet, the handler receives an existing session. A local command-line application normally constructs its own session. These examples must not be conflated when copied into production code.

Failure mode: hidden extraction
An engineer writes a clean-looking function that calls `to_pandas()`, then silently performs all grouping in local memory. Queries can appear fast in a six-row demonstration and later fail because the result is too large. Inspect the data movement as carefully as the formula.

Check. Which step transfers data: `group_by(...)` or `collect()`?

9.1.3 Choosing SQL, Snowpark, or external compute

A technology choice should be derived from the location of data, type of operation, governance boundary and operational ownership. SQL remains the clearest expression for many set-oriented joins, aggregations and window transformations. Snowpark is valuable when Python functions help assemble many consistent relational plans or when approved server-side Python algorithms are necessary. External compute is justified when a specialised library, runtime or integration cannot be deployed appropriately within Snowflake, but it increases the burden of moving data safely and proving reproducibility.

Choice	Strong fit	Common downside
SQL model	Stable declarative transformation	Complex Python control logic awkward
Snowpark DataFrame	Composed, reusable relational code	Additional Python dependency and tooling
Python UDF / procedure	Specialised logic near stored data	Runtime limits, packaging, row-level overhead
External Python	Custom integrations and runtimes	Extraction, network, identity, observability

Example. Convert unit-labelled synthetic meter readings to MWh with a simple expression. Writing this as an external Python loop would move data for no business reason. If the transformation instead requires a proprietary simulation package unavailable in the managed runtime, local or other cloud compute may be justified after a security and cost review.

Listing 3: A readable SQL baseline for equivalent Snowpark logic.

```
SELECT interval_utc, SUM(energy_mwh) AS total_mwh
FROM energy_silver
WHERE operation = 'UPSERT' AND energy_mwh >= 0
GROUP BY interval_utc;
```

An engineering comparison must hold the workload constant: identical input, warehouse configuration, output schema, cache conditions and concurrency. Do not claim “Snowpark is faster” or “SQL is cheaper” from a language label. If both interfaces lower to a similar SQL plan, the observed difference may come from action choices, input volume or collection to the client rather than fundamental engine speed.

Decision record exercise
For the Finland dashboard, write three sentences: the computation to perform; the execution boundary you choose; and how a reviewer can verify whether data ever leaves Snowflake. Include why the rejected options are less appropriate for this workload.

Transition. Once an interface is chosen, its credentials and session lifecycle become part of the production design, not a notebook afterthought.

9.2 Python sessions and connection management

9.2.1 Session creation and explicit options

A Snowpark `Session` represents the authenticated context used to submit statements, obtain DataFrames and manage certain client-side integrations. Local applications create and close sessions; some managed execution environments provide the session. Keep creation in the program's entry point and pass the session to pure transformation functions. Do not create a new connection inside each mapping function or hold a global authenticated session in a library imported by tests.

Listing 4: Connection configuration is injected, never hard-coded.

```
from snowflake.snowpark import Session

def run(config: dict):
    session = Session.builder.configs(config).create()
    try:
        return session.sql('SELECT CURRENT_ROLE()').collect()
    finally:
        session.close()
```

9.2.2 Authentication, role, and warehouse selection

Use approved authenticator methods such as interactive authentication for development, or an organisation-approved noninteractive identity for automation. Credentials belong in a secret manager, protected environment or supported connection configuration, *not* in a manuscript, example CSV or committed file. Select the least-privilege role and a dedicated, cost-limited warehouse. A developer should not use a powerful administrative role for routine reads and transforms.

Preflight query, run only with permission
Record `CURRENT_ACCOUNT()`, `CURRENT_ROLE()`, `CURRENT_WAREHOUSE()` and `CURRENT_DATABASE()` before a cloud test. A successful login does not imply permission to create permanent procedures or write into analytical schemas.

9.2.3 Connection shutdown and security controls

Always release a locally owned session in a `finally` block or a documented application lifecycle. Never log tokens, key material, connection strings with secrets or raw personal records. Decide how concurrent runs isolate sessions and statement timeouts. A retry after an authentication failure should not repeatedly disclose credentials or exhaust a login rate limit. Where code is executed in a Snowflake-hosted worksheet, use the injected session instead of duplicating connection setup.

Project reference: P095: Snowpark Python application template. *Not independently executed; rights review pending.*

Project reference: P096: Snowflake SQLAlchemy integration. *Not independently executed; rights review pending.*

Check. What closes the session if a query raises an exception, and who grants permissions to the service role?

9.3 DataFrames, expressions, and lazy evaluation

9.3.1 DataFrames as composable relations

A Snowpark DataFrame is best understood as a description of a relation and its transformations, not a snapshot of rows stored in Python memory. Its schema names, expression types and semantics correspond to Snowflake operations. A `select()` returns a new DataFrame; it does not mutate the existing DataFrame object. This functional style makes it natural to construct reusable functions that accept a DataFrame and return an enhanced DataFrame.

Listing 5: Composable transformations: no output action yet.

```
from snowflake.snowpark.functions import col, lit

def approved_energy(df):
    return (df.filter(col('OPERATION') == lit('UPSERT'))
              .filter(col('ENERGY_MWH').is_not_null())
              .filter(col('ENERGY_MWH') >= lit(0))
              .select('PROVIDER', 'TECHNOLOGY',
                      'INTERVAL_UTC', 'ENERGY_MWH'))

base = session.table('ENERGY_SILVER')
filtered = approved_energy(base)
```

Important precondition. The incoming table is already a *conformed latest-state table*. If it still contains old revisions, an `OPERATION='UPSERT'` filter will count obsolete versions. Chapter 8 established how to resolve revision order and tombstones before an aggregate is exposed. State this precondition explicitly in both code comments and data contracts.



Figure 9.2. Transformations build a query plan; an action triggers execution.

Schema convention
Prefer stable uppercase unquoted warehouse identifiers in the introductory lab; document exact casing and quoted-name exceptions for actual systems. Never rely on an accidental join column name or implicit cast without a schema assertion.

Project reference: P097: Snowpark data engineering introductory demo. *Not independently executed; rights review pending.*

Check. Why should `approved_energy` return a DataFrame rather than call `collect()` within itself?

9.3.2 Lazy evaluation and action triggers

Lazy evaluation separates the definition of a transformation from its actual evaluation. This allows a pipeline to reuse a shared filter and select only the columns required by a downstream query. It also creates a trap: a developer can mistake a constructed DataFrame for a completed side effect. For example, `session.sql('CREATE TABLE ...')` produces a DataFrame object; the command is submitted when an action such as `collect()` executes it.

Listing 6: Actions are deliberate execution boundaries.

```
source = session.table('ENERGY_SILVER')
valid = source.filter(col('ENERGY_MWH') >= 0)
print(valid.count())              # Executes a count query
valid.limit(3).show()             # Executes and displays
rows = valid.limit(3).collect()   # Moves at most 3 rows
# valid.collect()                  # Avoid if dataset unbounded
```

Correctness risk. Evaluating the same lazy plan at different times can observe different committed source data. A DataFrame does not always freeze a table at the instant the Python variable is created. When a pipeline requires repeatable reconciliation, define a stable input snapshot, staging boundary or explicit temporal query. Do not compare two queries performed minutes apart and assume their source versions matched.

Cost risk. Three actions may run three queries, not one cached computation. A quick `count()` for a debugging message may cause a large scan. Instrument every action point and distinguish exploratory diagnostics from tasks required by the production contract.

Practical action checklist
Before calling an action, ask: How many rows can return? Which account and warehouse executes the statement? Which source snapshot is visible? Is this an essential result or merely a debug print? Could it be replaced by a smaller deterministic assertion or an aggregate?

Design alternative. If a consumer only needs a total, return an aggregate DataFrame; do not collect the base data and sum it in Python. If a downstream pipeline must persist the result, use an intentional approved write path and verify its committed contents, not simply the existence of the plan object.

Project reference: P098: Snowpark Python official source. *Not independently executed; rights review pending.*

Check. If a script prints the first five rows twice, what additional work might the script be requesting from Snowflake?

9.3.3 Inspecting execution plans and generated SQL

A DataFrame pipeline is readable only if a reviewer can connect Python expressions to work performed by the database. Snowpark offers plan-inspection functionality, including `DataFrame.explain()` and query-generation interfaces that may vary with API release. These are *diagnostics*, not guarantees of a particular optimiser plan or runtime. Use Query History and Query Profile after an authorised cloud execution to investigate actual scans, pruning and compute.

Listing 7: Development-only plan inspection.

```
from snowflake.snowpark.functions import col
q = (session.table('ENERGY_SILVER')
     .filter(col('ENERGY_MWH') >= 0)
     .select('TECHNOLOGY', 'ENERGY_MWH'))
q.explain()  # Displays an execution-plan representation
```

Query inspection must distinguish generated SQL text from the physical execution plan selected by the service. A logical join may be reordered; a filter may be pushed down; caching may eliminate some work on a repeated query. A plan printed on an earlier API release or warehouse configuration is not a permanent contract.

For reproducible investigation, record a query tag, warehouse name and size, account edition, input row count, data profile and query ID. When comparing two plans, use the same data version and logical output. Comparing a cached aggregate against an uncached full-table export teaches little about language choice.

Evidence	What it can support
Python code review	Intended filter, join and projected columns
Generated SQL / explain	How a DataFrame expresses a query
Query Profile / History	Actual operator behaviour and billed execution context
Output assertions	Business correctness for the tested fixture

Diagnostic discipline
Start from the incorrect result, trace the data contract and output assertions, then inspect its physical plan when performance is the problem. Optimising a broken join can make the wrong total arrive more efficiently.

Project reference: P099: dbt Labs Python/Snowpark Formula 1 example. *Not independently executed; rights review pending.*

Check. What evidence would distinguish a costly row export from a well-pushed-down aggregation?

9.4 Transformations, joins, and aggregations

9.4.1 Filtering, joining, and grouping

The energy case requires a technology-to-category lookup. A left join preserves unmatched technologies for quality review; a strict inner join could silently discard facts when a lookup value is missing. Verify that the dimension contains exactly one row per TECHNOLOGY, otherwise a many-to-many join can multiply MWh totals. Group only after accepted latest-state data have been selected.

Listing 8: Core Snowpark plan used by the companion project.

```
from snowflake.snowpark.functions import col, sum as sf_sum
fact = session.table('ENERGY_SILVER')
dim  = session.table('TECHNOLOGY_DIM')
report = (fact.filter(col('OPERATION') == 'UPSERT')
          .join(dim, ['TECHNOLOGY'], 'left')
          .group_by('CATEGORY', 'INTERVAL_UTC')
          .agg(sf_sum('ENERGY_MWH').alias('TOTAL_MWH')))
```

9.4.2 Data types and casting checks

Treat quantity as a numeric type with suitable precision; record units explicitly. A string `'18.000'` is not automatically equivalent to a correctly validated measurement in every transformation. Use deliberate type conversion and rejection rules before an aggregate. Timestamps should represent explicit UTC source instants; local Finnish wall-clock reporting belongs in the presentation/model layer with daylight-saving behaviour specified.

9.4.3 Validation of transformation output

For the synthetic fixture, approved values are wind 14, solar 6, hydro 18, nuclear 16, bioenergy 8 and other 4 MWh, totalling **66.000 MWh**. Under the sample dimension, Renewable contains wind, solar, hydro and bioenergy (46 MWh); OtherLowCarbon contains nuclear (16 MWh); and Other contains the remaining 4 MWh. Those names are *teaching categories*, not an official energy-sector classification.

Output contract
Assert nonnegative measures, one current business fact per key, no unrecognised category, six visible latest-state rows, three expected category groups, and a conserved total of 66.000 MWh. A successful API call is not a passing business test.

Project reference: P100: Snowpark Python demonstrations. *Not independently executed; rights review pending.*

Check. Which join mistake doubles the total even when every source measurement is valid?

9.5 User-defined functions and stored procedures

9.5.1 UDFs, UDTFs, and stored procedures

A *scalar user-defined function* (UDF) returns a scalar value for its input; a *user-defined table function* (UDTF) can produce multiple rows; a *stored procedure* encapsulates a callable operation with control flow and, where authorised, data-management effects. These tools are useful, but not interchangeable. A built-in SQL expression should be preferred when it is sufficiently expressive, easier to optimise, and requires no bespoke Python runtime.

Listing 9: Pure handler idea; deployment and account rights not demonstrated.

```
def normalise_label(value: str) -> str:
    return (value or '').strip().upper()
# Register as a UDF only after testing and a deployment review.
```

9.5.2 Execution environments and appropriate use

A Python UDF is appropriate for a deterministic, small transformation not already well supported by SQL functions. Stored procedures are better suited to multi-step application logic requiring a defined invocation and structured error reporting. A UDTF is appropriate for specialised row expansion. Consider supported runtime versions, execution permissions, scaling characteristics and the possibility that Python boundary overhead dominates a simple SQL expression.

9.5.3 Permissions, packaging and limitations

The deployment role may need permissions to create functions or procedures and use an internal stage for permanent handler artefacts. Installed packages must be available through supported mechanisms; an arbitrary `pip install` on a local laptop does not make a dependency available inside a managed Python handler. External network access, secrets and memory behaviour require feature-specific configuration and policy review. For regulated datasets, deliberately review whether a handler may process raw inputs.

Decision checklist
Before registering Python code, document: output determinism; handling of nulls and exceptions; type contract; runtime/package versions; data-access and external-access permissions; test evidence; migration path; and fallback when a package version becomes unavailable.

Project reference: P101: UDF and UDTF guide examples. *Not independently executed; rights review pending.*

Project reference: P102: External functions examples. *Not independently executed; rights review pending.*

Check. Why is a procedure not simply a UDF with a longer Python file?

9.6 Dependencies, packaging, and code organisation

9.6.1 Dependency pinning and compatibility

The same notebook can behave differently months later if Snowpark or its transitive dependencies change. Keep a versioned package specification, lock exact versions before a reproducible publication release, and test against a declared Python interpreter. Constraints such as `>=1.18,<2` are a *development range*, not a final reproducibility lock; the distribution of this book's eventual companion artefacts must include resolved version records.

Listing 10: Project metadata; optional integrations are not installed for offline tests.

```
[project]
name = "snowflake-chapter09-energy-lab"
version = "0.1.0"
requires-python = ">=3.10"
dependencies = []
[project.optional-dependencies]
cloud = ["snowflake-snowpark-python>=1.18,<2"]
```

9.6.2 Modular Python project layout

Separate *business rules* (version ordering, time grain, units and invalid-data policy) from *connectors* (session credentials), *query builders* (Snowpark DataFrames), *application entry points* and *tests*. Prefer functions with explicit DataFrame inputs and outputs. Avoid notebooks that rely on hidden cell order, global state or personal file paths. The companion project includes `snowpark_pipeline.py`, `test_snowpark.py`, `data/` fixtures and optional SQL bootstrap templates.

9.6.3 Environment and artefact management

Never commit credentials, downloaded personal data or run logs with secrets. Record artefact hashes where they matter, link tests to tagged source revisions, and document how to recreate the environment from a clean checkout. For deployed procedures, treat source code, package manifest and stage artefacts as a controlled release rather than an ad hoc click sequence.

Reproducibility note
The printed text demonstrates a stable idea. The companion archive records the specific code and fixture used for the 22 October 2026 editorial check. A later edition should retest all feature-dependent statements and update its release tag rather than quietly changing the archive behind a printed claim.

Project reference: P103: Snowpark on Jupyter reference. *Not independently executed; rights review pending.*

Project reference: P015: Snowflake Python API reference. *Not independently executed; rights review pending.*

Check. What additional artefact is needed to make a permissive package constraint reproducible?

9.7 Local development and automated testing

9.7.1 Unit tests independent of live services

Most energy-data rules can be proved without a Snowflake account: a revision must supersede an older revision, a duplicate event must not double-count, a tombstone must remove a visible fact, and a negative MWh reading must be rejected. Pure Python contract tests are fast, deterministic and free of cloud consumption. They do not establish that the Snowpark DataFrame API correctly generates the intended SQL or that account permissions are sufficient.

Snowflake also documents a *local testing framework* for a subset of Snowpark DataFrame APIs. Its emulation is useful for earlier feedback but does not implement every function or exactly reproduce all warehouse behaviour. For example, the documentation distinguishes unsupported `Session.sql()` operations in local testing. Consequently, label the exact test type in the report rather than calling every successful offline test a Snowflake integration test.

9.7.2 Integration tests in a test account

A real integration check creates a small synthetic fixture under a disposable schema, builds the Snowpark plan and submits a deliberate action. Assert schema and numeric results, log the query ID, and remove only objects created by that test run. Repeat in the account edition and region relevant to readers if a feature varies by deployment. A test that succeeded for a six-row mock cannot prove the engine's throughput or package deployment limitations.

9.7.3 CI secrets and expensive-test controls

Split CI into mandatory offline tests and permission-gated cloud tests. Never enable cloud integration on every untrusted pull request, and never expose a high-privilege password to forked code. Gate paid tests behind an approval step, short-lived credentials, spending and timeout controls, and an auditable cleanup mechanism.

Listing 11: The reproducible default is an offline test.

```
python snowpark_pipeline.py
python -m unittest -v test_snowpark.py
# Live integration: deliberately not automated by default.
```

Project reference: P104: Testing Snowpark tutorial. *Not independently executed; rights review pending.*

Check. Which two assertions cannot be proved by an offline business-rule unit test alone?

9.8 Snowpark performance and execution trade-offs

9.8.1 Data movement and execution locality

A central aim of Snowpark is to let developers describe set-based work without extracting the entire dataset into a Python process. Consider two implementations of a regional generation total. Implementation A selects several million rows into pandas and groups locally. Implementation B groups by region in a Snowpark DataFrame and collects only the dozen rows of resulting totals. Both may produce correct MWh figures for a fixture; their network transfer, privacy exposure and client-memory demands are radically different.

Listing 12: Prefer a bounded aggregate over collecting all facts.

```
from snowflake.snowpark.functions import sum as sf_sum
summary = (session.table('ENERGY_SILVER')
           .group_by('TECHNOLOGY')
           .agg(sf_sum('ENERGY_MWH').alias('MWH')))
small_result = summary.collect()  # bounded by technology count
```

Not an unconditional guarantee. An aggregate can still be expensive within Snowflake, especially with poor clustering, excessive cardinality or skew. The benefit is that the client receives only the result, not every source row. If grouping by an unconstrained free-text user ID, the returned result might again be huge; boundedness is a domain assertion that the test must verify.

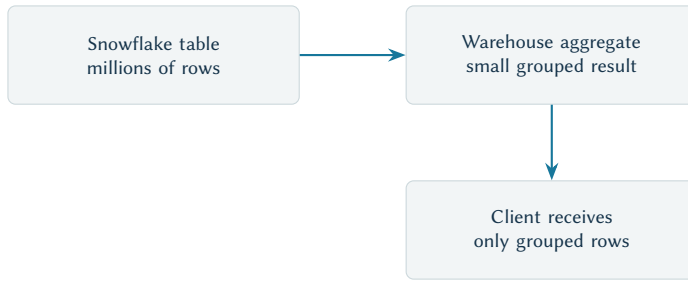


Figure 9.3. Push relational work to compute close to storage; transfer only necessary output.

Operational boundary
Before an application collects a DataFrame, define an expected maximum number of output groups, a query timeout and a failure response. If the resulting cardinality is unexpectedly large, alert and stop rather than serialising unreviewed records to a local file.

Check. Which implementation exposes the larger volume of raw data outside Snowflake, and how would you prove it?

9.8.2 Cost and latency comparisons without misleading benchmarks

Snowflake compute consumption depends on the services and resources actually used, not on whether an author writes `.group_by()` or `GROUP BY`. A benchmark must control the input snapshot, output equivalence, warehouse size, concurrency and cache conditions. It must also separate client time, queueing, execution duration, data-transfer time, Python handler overhead and billed credits. Without that breakdown, a wall-clock speed comparison encourages incorrect architectural conclusions.

The energy dataset in this book is intentionally tiny: twelve synthetic deliveries and six latest-state rows. Its measured local test runtime is not meaningful evidence about production-scale warehouse latency. To compare at scale, construct an approved, suitably sized synthetic workload and publish repeatable queries, query IDs, actual credit consumption and variance across runs. A fabricated price estimate is not a benchmark.

Measurement	Required context
Wall-clock latency	Start/end boundary, network, queueing, repetitions
Credits consumed	Warehouse and service category, idle time, time window
Rows scanned / output	Input version, predicates, pruning, group cardinality
Result correctness	Exact comparison against a reference oracle
Transfer volume	Bytes and number of records returned to client

A sound experiment might execute both SQL and Snowpark representations of the same approved hourly aggregate in a dedicated warehouse with controlled result caching. It should report *median and tail durations* only after collecting real measurements. Cost optimisation then targets the observed bottleneck: fewer repeated actions, narrower scans, efficient joins, or right-sized compute. Changing languages alone is not a validated strategy.

Benchmark release rule
Label every performance number as measured, simulated or illustrative. Store scripts, input generation method and raw run evidence. Never convert an unexecuted performance hypothesis into a production claim merely to make a chapter more persuasive.

Project reference: P018: Snowpark Python performance tips. *Not independently executed; rights review pending.*

Check. Why is it invalid to compare a warm-cache aggregate with a cold-cache full export?

9.8.3 Avoiding Python-level anti-patterns

Three common problems recur. **First, accidental eager collection:** an unbounded `collect()` or `to_pandas()` pulls records into a limited client process. **Second, row-at-a-time work:** repeated queries inside a Python loop add round trips and overhead where a set-based join would suffice. **Third, unnecessary UDFs:** wrapping built-in SQL arithmetic in Python can introduce runtime and deployment cost without changing business logic.

Listing 13: Avoid a query for every energy category.

```
# Anti-pattern (illustrative only):
# for name in thousands_of_categories:
#     session.table('ENERGY_SILVER').filter(
#         col('TECHNOLOGY') == name).collect()

# Better: one grouped relational plan.
summary = (session.table('ENERGY_SILVER')
           .group_by('TECHNOLOGY')
           .agg(sf_sum('ENERGY_MWH').alias('MWH')))
```

Optimisation does not license changing correctness requirements. If the source contains revisions, a smaller filter that discards a necessary tombstone might make a query faster while returning the wrong total. If a join's category dimension is not unique, aggregation may magnify every fact. Build tests for these risks before investigating operator-level tuning.

Another mistake is to rely on implicit ordering. Relational tables are unordered unless the query specifies an order. A preview sorted by event timestamp is a presentation choice, not a stable business resolution rule. Use explicit deterministic tie-breaking for revision events and an agreed metric grain.

Improve the system in this order
(1) Validate the business result; (2) inspect data movement; (3) measure actual query operators; (4) reduce redundant actions; (5) tune warehouse and table design; (6) rerun correctness tests. Document any trade-off that makes the code harder to reason about.

Project reference: P013: Pandas on Snowflake starting point. *Not independently executed; rights review pending.*

Check. How many remote queries are issued by a loop that performs one `collect()` per source identifier?

9.9 Production-oriented Python pipeline patterns

9.9.1 Reusable functions, configuration and logging

Production code should separate four kinds of responsibility: connectivity, query construction, validation and publication. A `build_report(session, ...)` function can return a lazy DataFrame. A `validate_contract(rows)` function can run entirely offline. An orchestrator chooses when to execute and how to persist data. Each responsibility receives explicit parameters rather than importing configuration from an undocumented notebook variable.

Logs should record release version, run ID, source identifier, row-count expectations, query ID, outcome, elapsed time and object names. They should *not* contain raw API secrets, passwords or unminimised personal records. A query tag can connect Snowflake Query History to a pipeline run, but it does not replace application-level data lineage or approved data retention.

Listing 14: An original, testable function boundary.

```
def build_category_report(session, source, dimension):
    """Return a lazy, read-only DataFrame plan."""
    from snowflake.snowpark.functions import col, sum as sf_sum
    fact = session.table(source).filter(col('ENERGY_MWH') >= 0)
    dim = session.table(dimension)
    return (fact.join(dim, ['TECHNOLOGY'], 'left')
            .group_by('CATEGORY', 'INTERVAL_UTC')
            .agg(sf_sum('ENERGY_MWH').alias('TOTAL_MWH')))
```

In this abbreviated illustration, the `source` must already be latest-state, with a single approved version per business key. Production callers should use a protected table name from allowlisted configuration rather than accept unchecked SQL identifiers from external requests.

Logging design
Persist a small JSON run ledger: input dataset version; code commit; query IDs; expected and observed row counts; aggregate checksum; error category; retry count; and final publication status. Distinguish *attempt failed* from *run committed*. This evidence supports both incident investigation and reader reproducibility.

Project reference: P105: Snowpark DevOps reference. *Not independently executed; rights review pending.*

Check. Which elements of an execution log are essential to reproduce a faulty run without printing sensitive data?

9.9.2 Retries, idempotency and cleanup

Retries are normal because transient failures occur: connections break, warehouses suspend, jobs time out and source services return errors. A retry is safe only if the intended work is idempotent or protected by an explicit transaction and publication boundary. For the energy example, replaying a previously accepted event must not create a second visible measurement. Higher revisions may change a result once; an older revision must not undo that correction.

Failure point	Safe recovery decision
Before query action	Rebuild plan from versioned inputs
After query, before record	Check committed target / query ID; never guess
Partial application output	Publish through atomic or versioned boundary
Duplicate delivery	Deduplicate stable event ID, preserve revision order
Uncertain third-party effect	Use external idempotency key / reconciliation

A Python exception does not necessarily prove that an earlier warehouse statement rolled back. If a network connection fails after a write completes, blindly replaying the write may duplicate data. The correct response is to inspect transaction and target-state evidence and to use deterministic `MERGE` or version-based publication patterns where they fit. This is the same operational lesson as the Streams/Tasks example in Chapter 8, now expressed through a Python-controlled process.

Cleanup should be explicit. Temporary output artefacts may be deleted by a local test after it has preserved required assertions. Cloud objects, however, may contain other users' data or active tasks. Destructive teardown statements should remain commented until a human confirms object ownership and permissions. Never hide production cleanup inside `finally` if failure diagnostics require the data to remain for investigation.

Idempotency invariant
For a fixed accepted event set, rerunning the pipeline must produce the same latest-state facts and exact MWh aggregate, regardless of repeated delivery. This is an observable property, not merely an implementation style.

Project reference: P106: Data engineering with Snowpark Python. *Not independently executed; rights review pending.*

Check. Why is "the Python function raised an exception" not sufficient proof that no database changes occurred?

9.9.3 Monitoring, versioning and production operations

An operational service needs signals for *liveness*, *freshness*, *correctness* and *expense*. Liveness asks whether a run finished. Freshness asks whether the latest approved interval is available within its agreed service objective. Correctness asks whether business invariants pass. Expense asks whether the observed credit and storage use remain inside a budget. A successful task status does not prove the energy aggregate is correct; conversely, a correct historical snapshot is not necessarily fresh enough for a dashboard.

Example monitoring contract (design targets, not measured data)
Expected latest source interval: explicit UTC; max desired end-to-end freshness: 30 minutes; visible business-key duplicates: zero; rejected negative readings: quarantined and reported; aggregate reconciliation: exact for the synthetic fixture; periodic cost review: consumption by pipeline and warehouse. These thresholds are proposals that operators must approve before production use.

Release management should bind the transformation logic, fixture, dependency lock, schema definition and test output to one version identifier. Schema drift and library upgrades require an explicit compatibility test. When stored procedures are involved, account runtime and package resolution can change deployability; a successful local `pip install` is not deployment proof.

A production readiness review should also ask: What happens if the *dimension table* is updated during a processing run? Can two jobs publish to the same destination at once? Is the source snapshot consistent? Are run IDs unique? Is the rollback strategy reversible? Is time-zone conversion delayed until a well-defined presentation layer? Which role owns the process when the original developer leaves?

Three questions that close the chapter
Where does this Python operation actually run? How would a new engineer prove that the latest energy value is the right one? What evidence would remain after a failed or expensive run? A maintainable Snowpark design makes all three answers inspectable.

Official technical references, inspected 22 October 2026: Working with DataFrames; local testing framework; stored procedures; Python UDFs. Recheck version-sensitive details prior to print release.

Project reference: P107: Snowpark data engineering and ML. *Not independently executed; rights review pending.*

9.L Guided lab: clean and enrich energy observations

9.L.1 Prerequisites, fixtures, permissions and cost guardrails

The companion archive provides `snowpark_pipeline.py`, `test_snowpark.py`, `pyproject.toml`, a synthetic CSV, optional SQL bootstrap and a review worksheet. The default path uses Python 3.10+ and the standard library; it needs no account, credentials, cloud access or paid compute. Optional Snowpark DataFrame execution requires an approved Snowflake account, a limited-role warehouse and compatible package versions. No such cloud execution is claimed in this edition.

The event fixture has **12 delivered rows**, including one duplicate delivery, one superseded wind measurement, one temporary TEST reading and subsequent tombstone, plus an invalid zero revision and negative solar reading. The model enforces explicit UTC, finite nonnegative quantities, event-ID deduplication and highest revision per provider/technology/interval key. This is a *synthetic* teaching dataset, not statistics from Fingrid or another external source.

9.L.2 Step-by-step transformation and evidence

Listing 15: Default reproducible steps from the lab directory.

```
python snowpark_pipeline.py
python -m unittest -v test_snowpark.py
```

The first command writes three files to `output/`; the second checks twelve invariants. The Snowpark helper accepts a Session and returns an *unexecuted* join-and-aggregation plan over approved source tables. It performs no write and is not imported by the offline tests.

Actually observed offline fixture result	Value
Delivered rows / invalid rows	12 / 2
Identical event deliveries skipped	1
Distinct accepted valid events	9
Visible latest-state facts	6
Latest-state tombstones	1
Approved energy aggregate (MWh)	66.000
Offline unit tests passed	12 / 12

Table 9.2. Deterministic Python result; no cloud-performance or Snowpark API test is implied.

Optional integration extension: Load exactly the six silver facts into a disposable table and a `TECHNOLOGY_DIM` with one row per technology, then call the lazy Snowpark plan. Expect three category groups and the same 66.000 MWh after grouping. Retain query IDs and environment versions; the optional route is *not executed* in this review.

9.L.3 Assertions, troubleshooting, rollback and cleanup

Twelve tests verify row accounting, six approved facts, the 66.000 MWh total, wind revision precedence, tombstones, duplicate/replay handling, malformed data, revision conflicts and UTC enforcement. They prove only the tested offline Python rules, not live Snowpark execution.

If the total differs, inspect the business key and revision order. If `TEST` reappears, preserve tombstones during latest-version selection. If category totals multiply, inspect dimension uniqueness. For cloud connection errors, inspect role, warehouse and policy without exposing secrets. A local mock limitation is not evidence of an engine bug.

Rollback and cleanup. The default artefacts are inside `output/`; preserve evidence, then delete that local folder. No cloud teardown is active. For optional integration, approve object ownership, removal and billing evidence.

9.L.4 Foundation, practitioner and production-design exercises

Foundation: Add a duplicated wind correction with the same `EVENT_ID`. Identify which counter changes and prove the approved total is unchanged. **Practitioner:** Add a later solar correction with a higher valid revision, then show both the new result and its invariance to replay; explain why arrival time is not the ordering key. **Production design:** Compare a single SQL model, a Snowpark DataFrame service and an external pandas service for a governed hourly dashboard. Specify data locality, role, test strategy, version pinning, latency SLO and teardown.

Assessment rubric (10 points)
Data grain and revision semantics (2); accurate Snowpark execution-boundary explanation (2); six-row and 66.000 MWh reconciliation (2); tests and replay handling (2); cost, least privilege, evidence and cleanup (2). Full credit requires stating what was not executed.

Chapter review. (1) What triggers a Snowpark DataFrame action? (2) Where can Python handlers execute? (3) How would you detect an exploding join? (4) What is the difference between a local mock test and a warehouse integration test? (5) Which operational evidence should accompany a production release?

Looking ahead. Chapter 10 places such transformations inside dbt models and repeatable ELT delivery workflows. Python does not remove the need for declared lineage, testable contracts, or principled production releases.

Print-release verification gate
Before print, recheck Snowpark runtimes and packages; execute and document a real Snowflake integration; substantiate any measured costs; and review external GitHub licensing.